LogisticsFlow

Notatki techniczne: PostgreSQL, SQLAlchemy, Alembic i tabela customers

Cel dokumentu: instrukcja krok po kroku, która pozwoli wrócić do wykonanej konfiguracji i odtworzyć najważniejsze czynności bez szukania ich w historii rozmowy.

1. Punkt wyjścia i cel

- Projekt: LogisticsFlow – backendowa aplikacja do obsługi zleceń transportowych.
- Cel tego etapu: przygotować PostgreSQL, połączyć aplikację z bazą przez SQLAlchemy i utworzyć tabelę customers za pomocą migracji Alembic.
- Wykonaliśmy i sprawdziliśmy połączenie z bazą oraz utworzenie tabeli. Nie oznacza to jeszcze, że endpointy API do obsługi klientów są gotowe.
- Katalog projektu używany w terminalu: ~/Dokumenty/Python_praktyczny/LogisticsFlow

2. Przygotowanie środowiska Python

2.1. Aktywacja środowiska wirtualnego

```
cd ~/Dokumenty/Python_praktyczny/LogisticsFlow
source venv/bin/activate
```

- Po aktywacji sprawdź, czy Python pochodzi z katalogu venv:

```
which python
which pip
```

- Oczekiwana ścieżka powinna zawierać LogisticsFlow/venv/bin/.

2.2. Instalacja bibliotek

```
python -m pip install sqlalchemy alembic "psycopg[binary]"
python -m pip install python-dotenv
```

- SQLAlchemy – ORM, czyli narzędzie do definiowania modeli i pracy z bazą przez obiekty Pythona.
- Alembic – narzędzie do wersjonowania zmian struktury bazy danych i wykonywania migracji.
- psycopg – sterownik, przez który Python komunikuje się z PostgreSQL.
- python-dotenv – odczytuje zmienne zapisane w pliku .env.

2.3. Zapis zależności projektu

```
python -m pip freeze > requirements.txt
```

- Polecenie zapisuje wersje zainstalowanych pakietów do requirements.txt.
- Używaj python -m pip, aby mieć pewność, że instalujesz pakiety do aktywnego interpretera.

3. Przygotowanie PostgreSQL

3.1. Sprawdzenie instalacji i działania

```
psql --version
sudo systemctl status postgresql
pg_lsclusters
```

- W tym projekcie sprawdziliśmy PostgreSQL 14.24.
- Klaster 14/main działał na porcie 5432 i miał status online.

3.2. Utworzenie użytkownika i bazy

- Zalogowaliśmy się do PostgreSQL jako administrator systemowy:

```
sudo -u postgres psql
```

- W konsoli psql utworzyliśmy osobnego użytkownika aplikacji. Hasło wpisuje się lokalnie – nie należy umieszczać go w kodzie ani wysyłać na GitHub.

```
CREATE USER logisticsflow_user WITH PASSWORD 'TWOJE_HASLO';
CREATE DATABASE logisticsflow OWNER logisticsflow_user;
```

- Jeśli baza została utworzona pod omyłkowo skróconą nazwą logisticsfl, zmieniliśmy jej nazwę:

```
ALTER DATABASE logisticsfl RENAME TO logisticsflow;
```

- Ważne: powyższe polecenie wykonuje się tylko wtedy, gdy baza faktycznie ma nazwę logisticsfl. Nie wykonuj go ponownie, jeśli baza nazywa się już logisticsflow.

3.3. Test połączenia użytkownika aplikacji

```
psql -h localhost -U logisticsflow_user -d logisticsflow
```

```
SELECT current_database(), current_user;
```

- Otrzymaliśmy potwierdzenie: baza logisticsflow, użytkownik logisticsflow_user.
- Wyjście z konsoli PostgreSQL: \q

4. Konfiguracja danych dostępowych

- Utworzyliśmy plik .env w głównym katalogu projektu.
- W .gitignore sprawdziliśmy wpis .env – dzięki temu plik z hasłem nie powinien być dodawany do repozytorium.

```
DATABASE_URL=postgresql+psycopg://logisticsflow_user:TWOJE_HASLO@localhost:5432/logisticsflow
```

- W miejsce TWOJE_HASLO wpisuje się lokalnie własne hasło.
- Jeśli hasło zawiera znaki specjalne, np. @, /, : lub #, trzeba je zakodować w adresie URL.
- Nigdy nie wklejaj zawartości .env do publicznego repozytorium ani do wiadomości.

5. Konfiguracja SQLAlchemy – app/database.py

Utworzyliśmy plik app/database.py. Odpowiada on za wczytanie konfiguracji, utworzenie silnika połączenia, przygotowanie sesji i bazowej klasy modeli.

```
import os

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase
from dotenv import load_dotenv

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL")

if not DATABASE_URL:
    raise RuntimeError("DATABASE_URL is not configured")

engine = create_engine(DATABASE_URL)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

class Base(DeclarativeBase):
    pass
```

5.1. Znaczenie elementów

- load_dotenv() – wczytuje zmienne z pliku .env.
- os.getenv('DATABASE_URL') – pobiera adres bazy ze zmiennej środowiskowej.
- create_engine() – tworzy silnik komunikacji SQLAlchemy z bazą; samo utworzenie engine nie musi jeszcze otworzyć połączenia.
- sessionmaker() – tworzy fabrykę sesji, z których aplikacja będzie korzystać do operacji na bazie.
- autocommit=False – transakcje nie są automatycznie zatwierdzane.
- autoflush=False – wyłącza automatyczne wysyłanie oczekujących zmian przed zapytaniami.
- bind=engine – wiąże tworzone sesje z naszym silnikiem.
- Base – klasa bazowa, po której dziedziczą modele ORM.

5.2. Test połączenia

Tymczasowo utworzyliśmy test_db.py w katalogu głównym projektu:

```
from app.database import engine

with engine.connect() as connection:
    print("Po■■czenie z PostgreSQL dzia■a!")
```

- Uruchomiliśmy: python test_db.py.
- Otrzymaliśmy komunikat: Połączenie z PostgreSQL działa!.
- Plik test_db.py był testem tymczasowym. Właściwe testy projektu będziemy później organizować z użyciem pytest.

6. Model SQLAlchemy – Customer

6.1. Struktura katalogów

```
app/
■■■ __init__.py
■■■ main.py
■■■ database.py
■■■ models/
    ■■■ __init__.py
    ■■■ customer.py
```

- Utworzyliśmy katalog app/models oraz pliki __init__.py i customer.py.

6.2. Kod modelu

```
from datetime import datetime, timezone

from sqlalchemy import String, Integer, DateTime
from sqlalchemy.orm import Mapped, mapped_column

from app.database import Base


class Customer(Base):
    __tablename__ = "customers"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    company_name: Mapped[str] = mapped_column(String(255), nullable=False)
    email: Mapped[str] = mapped_column(String(255), nullable=False)
    phone: Mapped[str] = mapped_column(String(20), nullable=False)
    contact_person: Mapped[str] = mapped_column(String(150), nullable=False)
    address: Mapped[str] = mapped_column(String(255), nullable=False)
    tax_id: Mapped[str] = mapped_column(String(10), unique=True, nullable=False)

    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        default=lambda: datetime.now(timezone.utc),
        nullable=False
    )
```

6.3. Znaczenie kolumn i ograniczeń

Kolumna	Typ / ograniczenia	Znaczenie
id	Integer, primary_key	Unikalny identyfikator rekordu; PostgreSQL generuje kolejne wartości.
company_name	String(255), NOT NULL	Nazwa firmy; wymagana, ale nie musi być unikalna.
email	String(255), NOT NULL	Adres e-mail; wymagany, bez ograniczenia UNIQUE.
phone	String(20), NOT NULL	Telefon jako tekst, bo może zawierać +, spacje i zera na początku.
contact_person	String(150), NOT NULL	Osoba kontaktowa po stronie klienta.
address	String(255), NOT NULL	Adres siedziby klienta.
tax_id	String(10), UNIQUE, NOT NULL	NIP wymagany i unikalny.
created_at	DateTime(timezone=True), NOT NULL	Data i czas utworzenia rekordu ze strefą czasową.

- Mapped[str] lub Mapped[int] – adnotacja typu Pythona używana przez SQLAlchemy ORM.
- mapped_column() – definiuje kolumnę tabeli.
- nullable=False – kolumna nie może przyjąć wartości NULL; nie oznacza unikalności.
- unique=True – baza nie pozwoli na powtórzenie wartości w tej kolumnie.
- String – tekst. Numer telefonu i NIP przechowujemy jako tekst, a nie liczby, ponieważ nie wykonujemy na nich obliczeń.
- default=lambda: datetime.now(timezone.utc) – funkcja zostanie wywołana przy tworzeniu rekordu, więc każdy klient otrzyma aktualny czas. Zapis datetime.now(timezone.utc) bez lambda wykonałby się wcześniej, podczas ładowania modułu.
- Ograniczenie: unikalność NIP-u nie sprawdza, czy NIP jest formalnie poprawny. Walidację trzeba dodać osobno, np. w schemacie Pydantic.

6.4. Rejestracja modelu

```
# app/models/__init__.py
from app.models.customer import Customer
```

- Import Customer ładuje model i rejestruje jego tabelę w metadanych Base. Dzięki temu Alembic może ją wykryć.

7. Inicjalizacja i konfiguracja Alembica

7.1. Inicjalizacja

```
python -m alembic init migrations
```

- Polecenie utworzyło katalog migrations/, podkatalog migrations/versions/, plik migrations/env.py, szablon migracji i alembic.ini.

7.2. Konfiguracja migrations/env.py

W env.py zaimportowaliśmy Base i Customer, wczytaliśmy .env i ustawiliśmy metadane modeli:

```
import os
from dotenv import load_dotenv

from app.database import Base
from app.models import Customer

load_dotenv()

# po config = context.config:
database_url = os.getenv("DATABASE_URL")

if not database_url:
    raise RuntimeError("DATABASE_URL is not configured")

config.set_main_option(
    "sqlalchemy.url",
    database_url.replace("%", "%%")
)

target_metadata = Base.metadata
```

- Import Customer jest potrzebny, aby model został załadowany i zarejestrowany w Base.metadata.
- target_metadata = Base.metadata przekazuje Alembicowi metadane tabel SQLAlchemy.
- Adres bazy jest pobierany z .env, a nie przechowywany z hasłem w alembic.ini.
- replace('%', '%%') zabezpiecza znaki procenta przed interpretacją przez parser konfiguracji Alembica.

7.3. Ważna lekcja dotycząca uruchamiania Alembica

- Samo polecenie alembic uruchomiło globalną instalację z ~/.local/bin/alembic, która nie widziała python-dotenv z venv.
- Sprawdziliśmy which python i which alembic. Python wskazywał na venv, ale alembic na ~/.local/bin.
- Rozwiązanie: uruchamialiśmy Alembica przez aktywny interpreter:

```
python -m alembic ...
```

8. Wygenerowanie i wykonanie migracji

8.1. Wygenerowanie pliku migracji

```
python -m alembic revision --autogenerate -m "create customers table"
```

- Alembic połączył się z PostgreSQL, porównał metadane modeli ze schematem bazy i wykrył dodaną tabelę customers.
- Wygenerował plik w migrations/versions/ o identyfikatorze 831de808085d.
- Przed wykonaniem migracji przejrzeliśmy wygenerowany plik. Zawierał create_table('customers'), kolumny, klucz główny id i UNIQUE dla tax_id.

8.2. Wykonanie migracji

```
python -m alembic upgrade head
```

- Polecenie zastosowało migrację w PostgreSQL. W terminalu pojawił się komunikat Running upgrade -> 831de808085d, create customers table.

8.3. Weryfikacja w PostgreSQL

```
psql -h localhost -U logisticsflow_user -d logisticsflow

\dt
\d customers
```

- Polecenie \dt pokazało tabele alembic_version i customers.
- Polecenie \d customers potwierdziło kolumny, typy danych, klucz główny customers_pkey, ograniczenie customers_tax_id_key oraz automatyczny generator id.
- created_at ma typ timestamp with time zone.
- Po zakończeniu pracy w konsoli psql wychodzimy poleceniem: \q.

9. Co oznaczają najważniejsze narzędzia?

Element	Rola w projekcie
PostgreSQL	Fizycznie przechowuje dane i strukturę tabel.
SQLAlchemy	Opisuje modele w Pythonie i umożliwia wykonywanie operacji na bazie przez ORM.
Alembic	Tworzy i wykonuje migracje zmieniające strukturę bazy w kontrolowany sposób.
.env	Przechowuje lokalną konfigurację, np. adres bazy i hasło.
python-dotenv	Wczytuje wartości z pliku .env.
psycopg	Sterownik komunikacji Python–PostgreSQL.
requirements.txt	Zapisuje zależności projektu i ich wersje.

10. Najważniejsze polecenia – ściąga

```
# Aktywacja ■rodowiska
source venv/bin/activate

# Sprawdzenie interpreterów
which python
which pip
which alembic

# Instalacja / aktualizacja zale■no■ci
python -m pip install sqlalchemy alembic "psycopg[binary]" python-dotenv
python -m pip freeze > requirements.txt

# Sprawdzenie PostgreSQL
psql --version
pg_lsclusters
sudo systemctl status postgresql

# Po■■czenie z baz■
psql -h localhost -U logisticsflow_user -d logisticsflow

# Alembic – zawsze przez aktywny interpreter
python -m alembic revision --autogenerate -m "opis migracji"
python -m alembic upgrade head

# PostgreSQL – w konsoli psql
\dt
\d customers
\q

# Git
git status
git log --oneline --graph
```

11. Aktualny stan i dalsze kroki

- Potwierdzone jako wykonane: PostgreSQL działa, baza logisticsflow istnieje, użytkownik aplikacji może się połączyć.
- Potwierdzone jako wykonane: SQLAlchemy engine i SessionLocal są skonfigurowane, a test połączenia zakończył się powodzeniem.
- Potwierdzone jako wykonane: model Customer został załadowany, migracja Alembic została wygenerowana i zastosowana.
- Potwierdzone jako wykonane: tabela customers istnieje w PostgreSQL, a jej kolumny i ograniczenia zostały sprawdzone.
- Do wykonania w kolejnym etapie: sprawdzić git status, dodać do repozytorium właściwe pliki, wykonać commit i push.
- Następnie: przygotować schematy Pydantic, endpointy CRUD dla klientów i testy API.
- Dalsze moduły projektu będą obejmować m.in. lokalizacje klientów, zlecenia transportowe, punkty załadunku/rozładunku, ładunki, kierowców, pojazdy i zdarzenia transportowe.

12. Krótka powtórka – sprawdź, czy pamiętasz

- 1. Które narzędzie opisuje tabelę w Pythonie? – SQLAlchemy.
- 2. Które narzędzie generuje i wykonuje migracje? – Alembic.
- 3. Gdzie trzymamy hasło do lokalnej bazy? – W .env, wykluczonym przez .gitignore.
- 4. Co oznacza nullable=False? – Wartość nie może być NULL.
- 5. Co oznacza unique=True? – Wartość nie może się powtarzać.

- 6. Dlaczego telefon przechowujemy jako String? – Może zawierać +, spacje i zera początkowe; nie wykonujemy na nim obliczeń.
- 7. Co robi alembic upgrade head? – Wykonuje migracje do najnowszej wersji.
- 8. Jak sprawdziliśmy tabelę w PostgreSQL? – Poleceniami \dt i \d customers.

Dokument przygotowany na podstawie kroków wykonanych w projekcie LogisticsFlow. Przy odtwarzaniu konfiguracji nie ujawniaj hasła z pliku .env i zawsze sprawdzaj, czy terminal korzysta z właściwego środowiska venv.